

Automated Air Traffic Control (ATC) Simulation & Testing Framework

A scoped, demo-realistic engineering plan for ASP.NET Core

Updated to reflect a 7-member team executing a 1.5-week (9-10 day) sprint, with an EF Core Database-First implementation as specified by the course coordinator.

GROUP 26 — TEAM ROSTER

IN26011871	Shriyam Rastogi
IN26012122	Arnav Majithia
IN26011732	Harsh Kumar Nimesh
IN26009582	Harsh Raj Singh
IN26009579	Aman Kumar Singh
IN26009732	Aditya Atreya
IN26011664	Aryaman Singh

Course: ASP.NET Development | Document status: Updated for realistic scope & delivery timeline
This revision supersedes the original aviation-grade specification with an achievable, demo-scale implementation that preserves every functional requirement, user story, and acceptance criterion in intent.

1. Position in the Project Lifecycle & Purpose of This Revision

This requirement analysis remains the precursor to system design, architecture, database schema, and sprint planning, as originally scoped. This revision exists because the original document specified parameters consistent with a production aviation safety system — 500 concurrent aircraft, sub-100ms real-time ticks, full WGS84 ellipsoidal geodesy, and live ADS-B ingestion. Those are appropriate targets for a funded, multi-month engineering effort, not for a 7-person team delivering a course project in 9-10 working days.

What changed: every functional requirement, non-functional requirement, actor, and user story from the original analysis is retained in this document. What has changed is the *scale of the constants* (aircraft count, tick rate, precision) and the *implementation approach* for components that would otherwise require infrastructure the team does not have time to build (live external feeds, a separately deployed agent service, production-grade uptime proof). Section 3 documents this mapping explicitly for transparency.

Core Objective: Establish unambiguous, achievable functional and non-functional requirements for a simulation framework that demonstrates safe testing of automated ATC conflict-resolution logic — built and demoed within the sprint window.

2. Business Problem & System Context

Current Challenge: Testing automated, algorithm-based ATC vectoring in live airspace is dangerous and non-viable. Existing flight simulators are typically disconnected from automated resolution engines.

Business Goal: Build a two-layer software framework (Simulation Engine + Control Layer) that ingests simulated flight telemetry, calculates flight kinematics on a fixed tick, detects horizontal/vertical separation conflicts, and tests automated tactical vectoring commands against those conflicts — all inside a single ASP.NET Core solution.

Technology Stack

BACKEND ASP.NET Core Web API (.NET 8)	REAL-TIME LAYER SignalR (WebSocket hub for radar push updates)
DATABASE SQL Server (LocalDB / Express)	ORM EF Core — Database-First (Scaffold-DbContext)
FRONTEND Razor Pages / lightweight JS + HTML5 Canvas radar view	BACKGROUND PROCESSING IHostedService / BackgroundService tick loop
TESTING xUnit (conflict-detection logic)	DISTANCE CALCULATION Haversine formula (spherical geodesy)

3. Scope Realignment: Original Specification vs. Sprint-Realistic Build

The table below maps every parameter that required scaling down, with the engineering rationale for each choice. This is intended to be shown directly during the presentation to demonstrate deliberate, justified scoping — not scope-cutting by accident.

Original Specification	Sprint-Realistic Implementation	Engineering Rationale
500 simultaneous aircraft, <100ms tick	15-30 aircraft, ~1 second tick	Still demonstrably real-time for a live demo; achievable in C# without load-testing infrastructure the team doesn't have time to build
WGS84 ellipsoidal geodesic precision to 0.01 NM	Haversine (spherical) distance formula	Matches the original document's own traceability matrix, which already specifies Haversine in ConflictDetector.cs; full ellipsoidal math is unnecessary precision for a simulation
Live ADS-B telemetry ingestion	Simulated JSON flight-state generator matching the ADS-B schema (ICAO24, callsign, lat/lon, altitude, heading, speed)	Real feeds require API keys, rate limits, and handling real-world data noise — out of scope for the timeline; the schema stays identical so a real feed could be substituted later
"Automated Controller Agent" as a separate external system actor	Internal C# service implementing an IControllerAgent interface within the same solution	A true external agent implies a second deployed service and API contract; an internal, interface-based implementation is swappable for a real external agent later without changing the rest of the system
99.9% uptime over continuous multi-hour runs	Demo-length runs (10-20 minutes), stability shown via one deliberate stress-test scenario	Uptime SLAs require infrastructure and monitoring the team cannot stand up in this timeframe, and no one will realistically verify this claim in a course demo

Presentation tip: keep this table as a dedicated slide. Framing the scope reduction as an explicit, justified engineering decision (rather than an omission) is what turns a limitation into a strength during evaluation.

4. Stakeholders

Stakeholder	Interest / Need
Aviation Software Researchers	Test and validate automated conflict-resolution algorithms in a safe, repeatable environment
ATC Systems Operators / Controllers	Supervise automated decisions via real-time radar views and issue manual overrides when necessary
Safety & Audit Analysts	Evaluate post-simulation safety metrics (minimum separation distances, reaction times)
System Administrators	Configure simulation parameters, airspace boundaries, and data feed connections

5. Actors

Actor	Role in the System
Algorithm / Safety Engineer	Sets up simulation scenarios, configures conflict rules, and runs stress tests
ATC Controller Supervisor	Views real-time radar visualization, monitors active separation alerts, issues manual overrides
Automated Controller Agent <i>(internal service, interface-based)</i>	Receives position updates and returns tactical flight vector commands
Flight Telemetry Generator <i>(simulated ADS-B-format feed)</i>	Produces live simulated aircraft position state vectors on each tick

6. Functional Requirements

A. Ingestion & Kinematics Layer

- FR-1.1** Ingest simulated JSON flight-state vectors (ICAO24, callsign, latitude, longitude, altitude, heading, speed) on each tick.
- FR-1.2** Update geographic coordinates and altitude per aircraft using standard kinematic formulas for each discrete time tick (Δt).

B. Conflict Detection & Management

- FR-2.1** Continuously calculate horizontal distance (Nautical Miles, via Haversine) and vertical separation (Feet) between all active aircraft pairs.
- FR-2.2** Trigger a Loss of Separation Alert when two aircraft violate minimum thresholds (< 5 NM horizontal **and** $< 1,000$ ft vertical).

C. Automated Vectoring & Resolution

- FR-3.1** Dispatch real-time aircraft state payloads to the Automated Controller Agent service.
- FR-3.2** Execute tactical vector commands returned by the controller (heading adjustment of $\pm 30^\circ$, climb/descend 1,000 ft).

D. Analytics & Supervision

- FR-4.1** Render an interactive 2D radar view (HTML5 Canvas) showing aircraft positions, tracks, and conflict alerts.
- FR-4.2** Record all simulation telemetry, conflict events, and reaction latencies to SQL Server for post-run evaluation.

7. Non-Functional Requirements

- NFR-1 Real-Time Performance:** the simulation loop must complete position updates and conflict calculations within one tick interval ($\sim 1s$) for up to 30 simultaneous aircraft.
- NFR-2 Accuracy:** distance and heading calculations use the Haversine spherical formula, consistent to standard aviation NM/ft units.
- NFR-3 Stability:** the simulation engine must remain stable for the full duration of a demo run (10-20 minutes) without crash or memory leak, verified by one dedicated stress-test scenario.
- NFR-4 Security & Control:** simulation configuration and manual override triggers are restricted to authenticated Supervisor-role users.

8. Agile User Stories & Acceptance Criteria

USER STORY 1 · CONFLICT DETECTION

As an Algorithm Engineer, I want the system to detect horizontal and vertical separation violations automatically, so that safety breaches can be caught before a collision occurs.

ACCEPTANCE CRITERIA

Given two aircraft flying at the same altitude (30,000 ft)

When the horizontal distance between them drops below 5 Nautical Miles

Then the system immediately triggers a ConflictAlert state

And transmits the conflict payload to the Automated Controller Agent

USER STORY 2 · AUTOMATED HEADING VECTORING

As an Automated Controller Agent, I want to send tactical heading and altitude commands to conflicting aircraft, so that safe separation is restored automatically.

ACCEPTANCE CRITERIA

Given an active ConflictAlert between Aircraft A and Aircraft B

When the ATC Agent issues a heading vector change command (e.g., turn left 30°)

Then the simulation engine updates Aircraft A's trajectory heading

And recalculates the predicted path on the next tick

USER STORY 3 · RADAR MAP VISUALIZATION

As an ATC Controller Supervisor, I want to see active aircraft positions and conflict zones on a visual radar map, so that I can visually inspect automated ATC decisions in real time.

ACCEPTANCE CRITERIA

1. The radar view renders position updates at a minimum of 1 frame per second.
2. Aircraft in conflict visually highlight in red.
3. Clicking an aircraft icon displays its callsign, current altitude, heading, and ground speed.

9. System Architecture

Single ASP.NET Core solution, three logical layers, communicating internally — no separately deployed services required for the sprint build.

```

| PRESENTATION LAYER |
| Razor Pages / JS + SignalR Client |
| (Canvas radar view, aircraft click -> detail panel) |
+-----+
| SignalR Hub (WebSocket) |
+-----+
| APPLICATION LAYER (ASP.NET Core) |
| +-----+ +-----+ |
| | Simulation | ---> | ConflictDetector.cs |
| | TickService | | (Haversine distance calc) |
| | (BackgroundService) | +-----+ |
+-----+
| v |
| +-----+ |
| | AutomatedControllerAgent |
| | implements IControllerAgent |
| | rule: conflict -> turn 30deg / +-1000ft |
| +-----+ |
+-----+
| EF Core (Database-First) |
+-----+
| SQL SERVER |
| SimulationRun | Aircraft | PositionLog |
| ConflictEvent | VectorCommand | AppUser |
+-----+

```

Data Flow (per tick)

1. Flight Telemetry Generator produces/advances aircraft state → TickService applies kinematics
2. ConflictDetector runs Haversine checks across all active aircraft pairs
3. On conflict: payload dispatched to AutomatedControllerAgent → returns a vector command
4. TickService applies the command to the aircraft's trajectory for the next tick
5. SignalR Hub broadcasts updated positions + conflict state to all connected clients
6. EF Core persists position log entries, conflict events, and issued commands to SQL Server

10. Database Design — EF Core Database-First

Per the course coordinator's instruction, the schema is authored directly in SQL Server first; EF Core models are then scaffolded from it via Scaffold-DbContext. No code-first migrations are used. The schema below is the authoritative source of truth for scaffolding.

SimulationRun		
Column	Type	Notes
RunId	INT, PK, Identity	Primary key
StartTime	DATETIME2	
EndTime	DATETIME2, NULL	Null while run is active
Status	VARCHAR(20)	Running / Completed / Aborted
CreatedByUserId	INT, FK → AppUser	

Aircraft		
Column	Type	Notes
AircraftId	INT, PK, Identity	Primary key
Callsign	VARCHAR(10)	
Icao24	VARCHAR(6)	
RunId	INT, FK → SimulationRun	

PositionLog		
Column	Type	Notes
PositionId	BIGINT, PK, Identity	Primary key
AircraftId	INT, FK → Aircraft	
Timestamp	DATETIME2	Tick timestamp
Latitude	DECIMAL(9,6)	
Longitude	DECIMAL(9,6)	
AltitudeFt	INT	
HeadingDeg	DECIMAL(5,2)	0–359.99
SpeedKts	INT	

ConflictEvent		
Column	Type	Notes
ConflictId	INT, PK, Identity	Primary key
RunId	INT, FK → SimulationRun	
AircraftAId	INT, FK → Aircraft	
AircraftBId	INT, FK → Aircraft	
DetectedAt	DATETIME2	
HorizontalDistNm	DECIMAL(6,3)	
VerticalDistFt	INT	
ResolvedAt	DATETIME2, NULL	
ResolutionAction	VARCHAR(50), NULL	

VectorCommand		
Column	Type	Notes
CommandId	INT, PK, Identity	Primary key
ConflictId	INT, FK → ConflictEvent	
AircraftId	INT, FK → Aircraft	
CommandType	VARCHAR(20)	Heading / Altitude
Value	DECIMAL(6,2)	Degrees or feet delta
IssuedAt	DATETIME2	

AppUser		
Column	Type	Notes
UserId	INT, PK, Identity	

Primary key

Username	VARCHAR(50), Unique	
PasswordHash	VARBINARY(256)	
Role	VARCHAR(20)	Supervisor / Engineer / Analyst

Database-First workflow: (1) author the six tables above directly in SQL Server Management Studio / Azure Data Studio with proper FK constraints → (2) run Scaffold-DbContext "connection-string" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models → (3) EF generates POCO entity classes and a DbContext automatically → (4) the team codes against the generated models. This satisfies the course requirement exactly and avoids any code-first migration usage.

11. Requirement Traceability Matrix

Business Need	User Story	Feature Module	Technical Implementation (.NET)
Prevent midair collisions	As an Engineer, detect separation breaches	Conflict Detector Service	ConflictDetector.cs — Haversine distance checks against 30 simulated aircraft
Automate tactical commands	As an Agent, issue heading adjustments	Controller Integration Engine	AutomatedControllerAgent.cs implementing IControllerAgent, handling vector commands
Real-time radar supervision	As a Supervisor, view active flights	Visualization Hub	SignalR WebSocket Hub + HTML5 Canvas frontend
Audit safety metrics	As an Analyst, review simulation logs	Event Logger	EF Core (Database-First) + SQL Server log persistence

12. Team Structure & Work Distribution

Work is distributed **by architectural layer** rather than by feature. All three user stories share the same core tick loop and conflict detector, so a feature-based split would create constant blocking between pairs working on the same shared files. A layer-based split lets all seven members work in parallel from day one against agreed interfaces.

Database & EF Core Lead

1 member

Owns the SQL Server schema design (Section 10) and Database-First scaffolding. Must finish schema design first — every other layer depends on the generated models. Also owns seed data for the demo scenario.

Simulation Engine & Conflict Detection

2 members

Owns the BackgroundService tick loop, kinematics updates, and ConflictDetector.cs (Haversine checks, threshold logic). This is the safety-critical core — paired for review, and the primary target for xUnit test coverage.

Automated Controller Agent

1 member

Implements IControllerAgent with rule-based resolution logic (conflict → heading/altitude command). Lightest layer in scope — can pair with the SignalR/API member if needed.

SignalR Hub & API Layer

1 member

Wires the simulation engine to SignalR broadcasts and exposes REST endpoints for run configuration, start/stop, and override triggers. Coordinates closely with both the engine team (data shape) and frontend team (client contract).

Frontend & Radar Visualization

2 members

Canvas-based radar rendering, SignalR client integration, aircraft click-through detail panel, conflict highlighting in red, and run-control UI (start/stop/override, event log view).

Sprint Timeline (9-10 working days)

Day 1-2	Schema design finalized and scaffolded (DB lead). All other members define stub interfaces/contracts in parallel so no one is blocked waiting on the database.
Day 3-5	Parallel build against mocked data: simulation engine, controller agent, SignalR hub, and frontend all developed independently.
Day 6-7	Integration — wire real EF-backed data through the full pipeline end to end; resolve interface mismatches.
Day 8	xUnit test coverage for the conflict detector, bug fixes, and construction of a reliable demo scenario with seeded conflicts.
Day 9-10	Polish, presentation slide preparation (including the scope-realignment slide), and full demo rehearsal.

Critical path: the database schema must not slip past Day 2. Since EF Database-First scaffolding requires the schema to exist in SQL Server before any models can be generated, a delay here cascades into every other layer. It is worth having the DB lead briefly align with the simulation engine pair on Day 1 to agree the schema together, avoiding rework once other members start coding against the generated models.

13. Requirement Analysis Sign-Off Checklist

- ✓ Business objective and collision-avoidance test objectives clearly defined
- ✓ User roles and actors (Engineers, Supervisors, Automated Agent) fully identified
- ✓ Functional scope — Ingestion, Kinematics, Conflict Detection, and Vectoring — documented
- ✓ Non-functional parameters scoped to sprint-realistic, demonstrable thresholds
- ✓ Acceptance criteria written in Given-When-Then format for all user stories
- ✓ Scope realignment from original specification documented with engineering rationale
- ✓ Database schema defined for EF Core Database-First scaffolding

✔ Work distributed across all 7 team members with a defined 9–10 day timeline